

Lecture 8.

모듈 활용하기



Ⅱ _ 모듈 활용하기



II 강의 목표

1 강의 목표

1. 모듈의 개념과 사용시 장점을 탐색
2. 모듈의 작성과 사용 방법 학습
3. `_name_` 전역변수에 대한 학습
4. 난수, 시간과 관련 기본 모듈 학습

II 빅데이터 처리를 위한 파이썬 프로그래밍

1 모듈의 개념

✓ 프로그래밍에서의 모듈

- 마트의 간편식처럼 프로그램을 개발할 때도 이미 만들어져 있는 코드를 가져다 쓸 수 있는 기능
- 즉 모듈이란 특정 기능(함수)을 포함하고 있는 파일(xxx.py)로 다른 프로그램에 이식해서 사용하는 것을 의미함



그림 12-1 이미 만들어져 있는 간편식

II 빅데이터 처리를 위한 파이썬 프로그래밍

1 모듈의 개념

✓ 프로그래밍에서의 모듈

- 함수를 학습할 때 특정 기능을 함수로 만들고 저장한 파일이 모듈임
- 즉 기능을 모아 함수가 되고 함수가 모여 모듈이 됨
- 모듈은 파이썬에서 기본적으로 제공하는 것도 있고, 개인 혹은 단체가 만들어서 배포하는 것도 있음

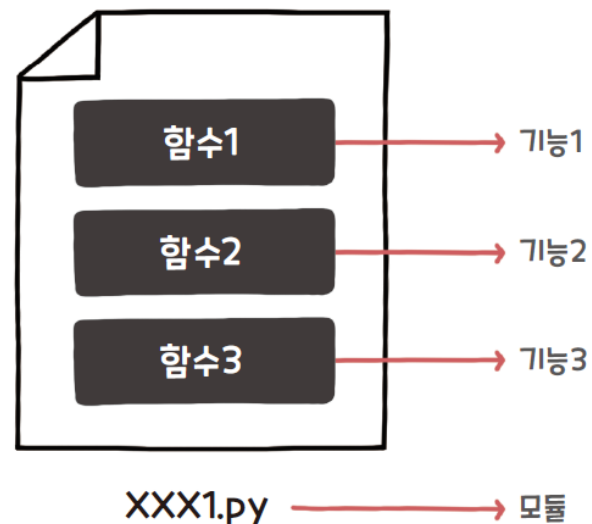


그림 12-2 함수의 모임 모듈

II 빅데이터 처리를 위한 파이썬 프로그래밍

1 모듈의 개념

✓ 모듈을 사용할 때 장점

- 프로그램 개발 시간을 단축시킬 수 있음
- 모듈은 이미 검증된 코드이므로 오류가 적음
- 팀원들이 기능 구현을 분업화하고 공유할 수 있어 전체적인 작업 속도를 향상시킬 수 있음

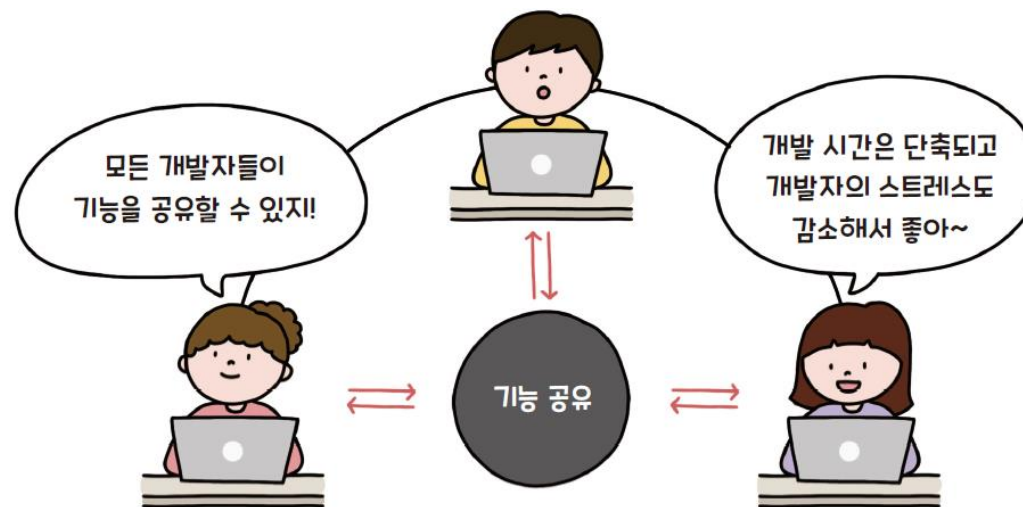


그림 12-3 모듈의 장점

II 빅데이터 처리를 위한 파이썬 프로그래밍

1 모듈의 개념

✓ 모듈을 사용할 때 장점

- 프로그램 개발 시간을 단축시킬 수 있음
- 모듈은 이미 검증된 코드이므로 오류가 적음
- 팀원들이 기능 구현을 분업화하고 공유할 수 있어 전체적인 작업 속도를 향상시킬 수 있음

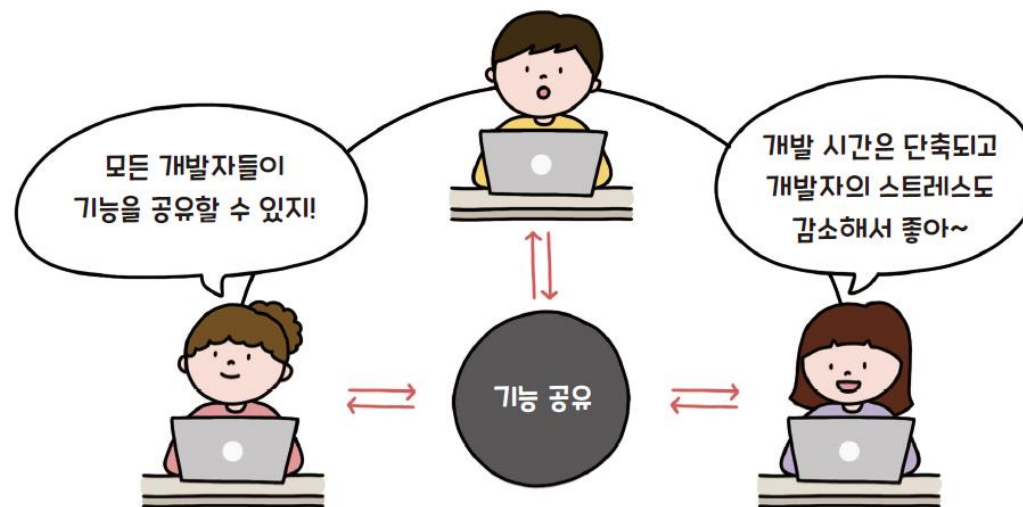


그림 12-3 모듈의 장점

II 빅데이터 처리를 위한 파이썬 프로그래밍

1 모듈의 개념

- ✓ Colab에서 모듈 만들기
- ✓ 일반 Python 환경에서 모듈 만들기

II 빅데이터 처리를 위한 파이썬 프로그래밍

1 모듈의 개념

- ✓ Colab에서 모듈 만들기 (아래의 코드를 작성)

  Untitled30.ipynb ☆ ☁

파일 수정 보기 삽입 런타임 도구 도움말

🔍 명령어 | + 코드 + 텍스트



```
1 def convertUnit(lenMm):
2
3     unitDic = {}
4     unitDic['cm'] = lenMm * 0.1
5     unitDic['m'] = lenMm * 0.001
6     unitDic['inch'] = lenMm * 0.03937
7     unitDic['ft'] = lenMm * 0.003281
8
9     return unitDic
10
11 def printLength(originData, lengths):
12     for len in lengths.keys():
13         print(originData, 'mm -->', lengths[len], len)
```

II 빅데이터 처리를 위한 파이썬 프로그래밍

1 모듈의 개념

✓ Colab에서 모듈 만들기

The screenshot shows the Google Colab interface. At the top, the file name 'test_convert.ipynb' is highlighted with an orange box. To its right, the text '1. 파일 이름 바꿔주기' is displayed. Below the file name, a dropdown menu is open, listing various actions. The '다운로드' (Download) option at the bottom of the menu is highlighted with an orange box. A secondary dropdown menu is visible next to '다운로드', showing two options: '.ipynb 다운로드' and '.py 다운로드'. The '.py 다운로드' option is also highlighted with an orange box. The background shows a code editor with some Python code.

1. 파일 이름 바꿔주기

2. .py 형태로 파일 다운 받기

II 빅데이터 처리를 위한 파이썬 프로그래밍

1 모듈의 개념

- ✓ Colab에서 모듈 만들기
 - Content > drive > MyDrive 내에 임의의 폴더 생성
 - 폴더 생성 후, 다운받은 .py 파일을 넣어주기
 - 예시의 경우 (2025_AI_Programming 폴더 내, test_convert.py 파일 넣기)

II 빅데이터 처리를 위한 파이썬 프로그래밍

1 모듈의 개념

- ✓ Colab에서 모듈 만들기
 - 새로운 노트북 실행
 - 아래의 코드 작성

✓
1초



```
1 from google.colab import drive
2 drive.mount('/content/drive')
3
4 import sys
5 sys.path.append('/content/drive/MyDrive/2025_AI_Programming')
6
7 import test_convert
8
```

II 빅데이터 처리를 위한 파이썬 프로그래밍

1 모듈의 개념

- ✓ Colab에서 모듈 만들기
 - 새로운 노트북 실행
 - 아래의 코드 작성

구글 드라이브 접근 권한 허용

✓
1초



```
1 from google.colab import drive
2 drive.mount('/content/drive')
3
4 import sys
5 sys.path.append('/content/drive/MyDrive/2025_AI_Programming')
6
7 import test_convert
8
```

구글 드라이브 접근 권한 허용된 폴더를
Path에 삽입

1

모듈의 개념

- ✓ Colab에서 모듈 만들기
 - 아래의 코드 작성

```
1 from google.colab import drive
2 drive.mount('/content/drive')
3
4 import sys
5 sys.path.append('/content/drive/MyDrive/2025_AI_Programming')
6 |
7 import test_convert
8
9 testDicValue = test_convert.convertUnit(0.1)
10
11 print(testDicValue)
12 print("-----")
13
14 testprintLength = test_convert.printLength(0.1, testDicValue)
15
```

↗ Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
{'cm': 0.010000000000000002, 'm': 0.0001, 'inch': 0.003937, 'ft': 0.0003281}

0.1 mm --> 0.010000000000000002 cm
0.1 mm --> 0.0001 m
0.1 mm --> 0.003937 inch
0.1 mm --> 0.0003281 ft

II 빅데이터 처리를 위한 파이썬 프로그래밍

1 모듈의 개념

- ✓ Colab에서 모듈 만들기
 - 모듈 import 결과 확인

```
1 def convertUnit(lenMm):
2
3     unitDic = {}
4     unitDic['cm'] = lenMm * 0.1
5     unitDic['m'] = lenMm * 0.001
6     unitDic['inch'] = lenMm * 0.03937
7     unitDic['ft'] = lenMm * 0.003281
8
9     return unitDic
10
11 def printLength(originData, lengths):
12     for len in lengths.keys():
13         print(originData, 'mm -->', lengths[len], len)
```

test_convert.py

```
1 from google.colab import drive
2 drive.mount('/content/drive')
3
4 import sys
5 sys.path.append('/content/drive/MyDrive/2025_AI_Programming')
6
7 import test_convert
8
9 testDicValue = test_convert.convertUnit(0.1)
10
11 print(testDicValue)
12 print("-----")
13
14 testprintLength = test_convert.printLength(0.1, testDicValue)
15
```

1

모듈의 개념

문제 해결 12-1

시험 통과 여부 출력하기

사용자가 3과목의 시험 점수를 입력하면 총점, 평균, 통과 여부를 출력하는 프로그램을 모듈을 이용해서 만들어봅시다. 시험 통과 여부는 평균이 60이상이고 한 과목도 40미만이 없다면 'Pass', 그렇지 않으면 'Fail'로 판정합니다.

모듈 파일

examCalcu\examCalculator.py

```
01 def getTotalScore(s1, s2, s3):
02     return s1 + s2 + s3
03
04 def getAverageScore(s1, s2, s3):
05     return getTotalScore(s1, s2, s3) / 3
06
07 def getPassFail(s1, s2, s3):
08
09     if (getAverageScore(s1, s2, s3) >= 60):
10         if(s1 >= 40 and s2 >= 40 and s3 >= 40):
11             return 'Pass'
12
13     return 'Fail'
```


II 빅데이터 처리를 위한 파이썬 프로그래밍

1 모듈의 개념

실행 파일

examCalcu\ch12_sol_01.py

```
01 import examCalculator
02
03 score1 = int(input('1과목 점수를 입력하세요. '))
04 score2 = int(input('2과목 점수를 입력하세요. '))
05 score3 = int(input('3과목 점수를 입력하세요. '))
06
07 print('총점 : ', examCalculator.getTotalScore(score1, score2, score3))
08 print('평균 : ', examCalculator.getAverageScore(score1, score2, score3))
09 print('합격여부 : ', examCalculator.getPassFail(score1, score2, score3))
```

```
1과목 점수를 입력하세요. 65
2과목 점수를 입력하세요. 39
3과목 점수를 입력하세요. 90
총점 : 194
평균 : 64.66666666666667
합격여부 : Fail
```

2

모듈의 활용

✓ as 키워드

- 모듈명이 길면 사용할 때마다 코드가 길어지는 불편함이 있음
- 예를 들어 calaulator 모듈의 함수를 불러오기 위해서는 매번 앞에 calculator를 붙여 사용해야 함

코드 12-1 모듈 파일

ex01\calculator.py

```
01 def addition(n1, n2):
02     print('덧셈 연산 결과 : ', n1 + n2)
03
04 def subtraction(n1, n2):
05     print('뺄셈 연산 결과 : ', n1 - n2)
06
07 def multiplication(n1, n2):
08     print('곱셈 연산 결과 : ', n1 * n2)
09
10 def division(n1, n2):
11     print('나눗셈 연산 결과 : ', n1 / n2)
12
13 def rest(n1, n2):
14     print('나머지 연산 결과 : ', n1 % n2)
15
16 def portion(n1, n2):
17     print('몫 연산 결과 : ', n1 // n2)
```

```
calculator.addition()
calculator.subtraction()
calculator.multiplication()
calculator.division()
calculator.rest()
calculator.portion()
```

모듈명을 매번 중복해서
앞에 붙여야 한다.

그림 12-6 긴 모듈명 사용에 따른 번거로움

II 빅데이터 처리를 위한 파이썬 프로그래밍

2 모듈의 활용

✓ as 키워드

- 'as' 키워드를 이용하면 모듈의 이름을 다른 이름으로 치환할 수 있음
- 'calculator'를 'ca'로 치환하면 기존의 'calculator' 대신 'ca'를 이용할 수 있음

코드 12-5 실행 파일

ex01\ch12_03.py

```
01 import calculator as ca    # calculator를 ca로 치환한다.
02
03 ca.addition(1, 2)
04 ca.subtraction(1, 2)
05 ca.multiplication(1, 2)
06 ca.division(1, 2)
07 ca.rest(1, 2)
08 ca.portion(1, 2)
```

치환된 **ca**를 이용해
calculator의 모듈을
불러온다.



calculator에게 ca라는
별명을 붙여주자!

II 빅데이터 처리를 위한 파이썬 프로그래밍

2 모듈의 활용

✓ as 키워드

- 'as' 키워드를 이용하면 모듈의 이름을 다른 이름으로 치환할 수 있음
- 'calculator'를 'ca'로 치환하면 기존의 'calculator' 대신 'ca'를 이용할 수 있음

코드 12-5 실행 파일

ex01\ch12_03.py

```
01 import calculator as ca    # calculator를 ca로 치환한다.
02
03 ca.addition(1, 2)
04 ca.subtraction(1, 2)
05 ca.multiplication(1, 2)
06 ca.division(1, 2)
07 ca.rest(1, 2)
08 ca.portion(1, 2)
```

치환된 **ca**를 이용해
calculator의 모듈을
불러온다.



calculator에게 ca라는
별명을 붙여주자!

II 빅데이터 처리를 위한 파이썬 프로그래밍

2 모듈의 활용

- ✓ from 키워드
 - 모듈의 전체 기능 중에서 일부 기능만 import할 수 있음
 - calculator 모듈에서 필요한 더하기, 빼기 기능만 가지고 오는 프로그램

코드 12-7 실행 파일

ex01\ch12_05.py

```
01  from calculator import addition      # calculator의 addition만 가져온다.
02  from calculator import subtraction  # calculator의 subtraction만 가져온다.
03
04  addition(1, 2)
05  subtraction(1, 2)
```

덧셈 연산 결과: 3

뺄셈 연산 결과: -1

II 빅데이터 처리를 위한 파이썬 프로그래밍

2 모듈의 활용

하나 더 알기

만약 import 하지 않은 기능을 사용하면 어떻게 되나요?

[코드 12-7]에서 addition과 subtraction 기능만 import 했기 때문에 multiplication() 또는 division()의 함수를 사용하면 에러가 발생합니다. 다음 [코드 12-8]은 multiplication()을 사용했을 때 발생하는 에러를 보여줍니다.

코드 12-8 실행 파일

ex01\ch12_06.py

```
01 from calculator import addition
02 from calculator import subtraction
03
04 addition(1, 2)
05 subtraction(1, 2)
06 multiplication(1, 2) ●———— import하지 않은 기능을 호출해서 에러가 발생한다.
```

덧셈 연산 결과 : 3

뺄셈 연산 결과 : -1

Traceback (most recent call last):

NameError: name 'multiplication' is not defined



II 빅데이터 처리를 위한 파이썬 프로그래밍

2 모듈의 활용

✓ 콤마(,)

- 모듈을 import할 때 콤마(,)를 이용해서 기능을 나열할 수 있음

```
from calculator import addition
from calculator import subtraction
```



```
from calculator import addition, subtraction
```

before

after

✓ 에스터리크(*)

- 모듈의 모든 기능을 가져 올 때는 에스터리크(*)를 이용함

```
from calculator import addition
from calculator import subtraction
from calculator import multiplication
from calculator import division
from calculator import rest
from calculator import portion
```

calculator에서 각각의 기능을 가져온다.



```
from calculator import *
```

calculator 모듈의
모든 기능을 가져온다.

before

after

II 빅데이터 처리를 위한 파이썬 프로그래밍

3 자주 활용하는 제공 모듈

✓ Math 모듈

- 수학 함수와 관련된 모듈
- 사칙연산보다 어려운 수식이 필요할 때 사용함

표 12-3 자주 쓰이는 math 모듈의 함수

함수	내용
<code>fabs()</code>	절댓값을 반환
<code>ceil()</code>	소수점 이하 올림한 값을 반환
<code>floor()</code>	소수점 이하 내림한 값을 반환
<code>trunc()</code>	소수점 이하 버림한 값을 반환
<code>gcd()</code>	최대공약수를 반환
<code>factorial()</code>	팩토리얼한 값을 반환
<code>sqrt()</code>	제곱근을 반환

II 빅데이터 처리를 위한 파이썬 프로그래밍

3 자주 활용하는 제공 모듈

하나 더 알기 Y

파이썬 내장 함수 중 수학과 관련된 함수

math 모듈은 아니지만 파이썬 내장 함수 중에서 유용한 수학 관련 함수도 있습니다.

표 12-4 수학 관련 내장 함수

함수	내용
sum()	전체 합을 반환
max()	최댓값을 반환
min()	최솟값을 반환
abs()	절댓값을 반환
pow()	거듭제곱을 반환
round()	반올림한 값을 반환



II 빅데이터 처리를 위한 파이썬 프로그래밍

3 자주 활용하는 제공 모듈

- ✓ random 모듈
 - 난수(random number)를 발생시키는 모듈
 - 복권 추첨, 주사위 던지기과 같이 임의의 숫자가 필요한 경우에 사용

표 12-5 자주 쓰이는 random 모듈의 함수

함수	내용
random()	0이상 1미만의 난수 발생
randint(n1, n2)	n1이상 n2이하의 난수 발생
randrange(n1, n2)	n1이상 n2미만의 난수 발생
sample(range(n1, n2), n3)	n1이상 n2미만의 난수 n3개 발생
choice()	무작위로 1개 아이템 선택함
shuffle()	아이템 순서 섞음

II 빅데이터 처리를 위한 파이썬 프로그래밍

3 자주 활용하는 제공 모듈

코드 12-18

ch12_10.py

```
01 import random
02
03 print('random : ', random.random())          # 0이상 1미만 난수
04 print('randint : ', random.randint(0, 9))     # 0이상 9이하 난수
05 print('randrange : ', random.randrange(0, 9)) # 0이상 9미만 난수
06 print('sample : ', random.sample(range(0, 10), 3)) # 0이상 10미만 난수 3개
07
08 listVar = [1, 2, 3, 4, 5, 6]
09
10 # 무작위로 1개 아이템 선택
11 print('choice : ', random.choice(listVar))
12 print('choice : ', random.choice(listVar))
13
14 # 아이템 순서 섞음
15 random.shuffle(listVar)
16 print('shuffle : ', listVar)
17 random.shuffle(listVar)
18 print('shuffle : ', listVar)
```

```
random : 0.39593355510791717
randint : 3
randrange : 3
sample : [1, 2, 6]
choice : 2
choice : 3
shuffle : [1, 6, 5, 4, 3, 2]
shuffle : [3, 2, 6, 1, 5, 4]
```



말 그대로 난수가 나오기 때문에
실행할 때마다 결과는 달라집니다.

II 빅데이터 처리를 위한 파이썬 프로그래밍

3 자주 활용하는 제공 모듈

✓ time 모듈

- 시간 관련 모듈
- 이 모듈은 UTC(Coordinated Universal Time)에서 정한 세계 표준시인 1970년 1월 1일 0시 0분 0초부터 초단위로 측정하는 것을 기준으로함
- 예전에는 그리니치 표준시(GMT)로 알려졌기 때문에 UTC와 GMT는 같은 시간을 뜻함

표 12-6 자주 쓰이는 time 모듈의 함수

함수	내용
localtime()	시스템의 현재 시간을 반환
gmtime()	GMT 시간을 반환
strftime()	시간 포맷 코드에 따른 출력

II 빅데이터 처리를 위한 파이썬 프로그래밍

3 자주 활용하는 제공 모듈

- ✓ localtime() 함수를 이용하여 현재 컴퓨터에서 받아온 시간을 보여줌

코드 12-19

ch12_11.py

```
01 import time
02
03 lt = time.localtime()           # 시스템 시간
04 print('localTime : ', lt)
05
06 # 상수를 이용한 시스템 시간 출력
07 print('tm_year : ', lt.tm_year)  ← 받아온 시스템 시간에서 년도만 뽑아낸다.
08 print('tm_mon : ', lt.tm_mon)    # 월
09 print('tm_mday : ', lt.tm_mday)  # 일
10 print('tm_hour : ', lt.tm_hour)  # 시
11 print('tm_min : ', lt.tm_min)    # 분
12 print('tm_sec : ', lt.tm_sec)    # 초
13 print('tm_wday : ', lt.tm_wday)  # 요일
```

```
localTime : time.struct_time(tm_year=2020, tm_mon=8, tm_mday=30, tm_hour=9, tm_min=21,
tm_sec=10, tm_wday=6, tm_yday=243, tm_isdst=0)
tm_year : 2020
tm_mon : 8
tm_mday : 30
tm_hour : 9
tm_min : 21
tm_sec : 10
tm_wday : 6
```

II 빅데이터 처리를 위한 파이썬 프로그래밍

3 자주 활용하는 제공 모듈

- ✓ 세계 표준시인 GMT 시간을 반환하는 gmtime() 함수를 이용한 코드

코드 12-20

ch12_12.py

```
01 import time
02
03 gt = time.gmtime()
04 print('time.gmtime() : ', gt)      # GMT 시간
05
06 # 상수를 이용한 GMT 시간 출력
07 print('tm_year : ', gt.tm_year)    받아온 GMT 시간에서 년도만 뽑아낸다.
08 print('tm_mon : ', gt.tm_mon)      # 월
09 print('tm_mday : ', gt.tm_mday)    # 일
10 print('tm_hour : ', gt.tm_hour)    # 시
11 print('tm_min : ', gt.tm_min)      # 분
12 print('tm_sec : ', gt.tm_sec)      # 초
13 print('tm_wday : ', gt.tm_wday)    # 요일
```



서울의 시간은
GMT+9입니다.

```
time.gmtime() : time.struct_time(tm_year=2020, tm_mon=8, tm_mday=30, tm_hour=0, tm_min=21, tm_sec=10, tm_wday=6, tm_yday=243, tm_isdst=0)
tm_year : 2020
tm_mon : 8
tm_mday : 30
tm_hour : 0
tm_min : 21
tm_sec : 10
tm_wday : 6
```

II 빅데이터 처리를 위한 파이썬 프로그래밍

3 자주 활용하는 제공 모듈

- ✓ strftime() 함수를 이용하여 시간을 표현한 코드
 - strftime()은 gmtime()이나 localtime()에 의해 나온 튜플이나 struct_time을 지정된 포맷을 통해 문자열로 변환함
 - 포맷은 소문자와 대문자의 뜻이 다르므로 구분하여 사용해야 함

코드 12-21

ch12_13.py

```
01 import time
02
03 print(time.strftime('%H:%M:%S', time.localtime()))
04 print(time.strftime('%Y:%m:%d:%H:%M:%S', time.localtime()))
05 print(time.strftime('%H:%M:%S', time.gmtime()))
06 print(time.strftime('%Y:%m:%d:%H:%M:%S', time.gmtime()))
```

```
09:21:10
2020:08:30:09:21:10
00:21:10
2020:08:30:00:21:10
```

II 빅데이터 처리를 위한 파이썬 프로그래밍

3 자주 활용하는 제공 모듈

문제

사용자가 1부터 45까지의 정수 6개를 입력하고, 컴퓨터에서는 1부터 45까지의 난수 6개를 발생합니다. 사용자 정수와 컴퓨터 정수를 비교해서 일치하는 숫자를 출력하는 프로그램을 만들어봅시다.



1부터 45까지의 정수 6개를 입력하세요.

Number 1 : 44

Number 2 : 45

Number 3 : 32

Number 4 : 1

Number 5 : 19

Number 6 : 22

이번주 로또 번호 [27, 1, 22, 2, 24, 19]

내가 선택한 번호 [44, 45, 32, 1, 19, 22]

일치하는 숫자 : [1, 19, 22]

3

자주 활용하는 제공 모듈

해결

모듈 파일

lottoMachine.py

```
01 import random
02
03 userNums = []
04 randNums = []
05 collect = []
06
07 def setUNumbers(ns):
08     global userNums
09     userNums = ns
10
11 def getUNumbers():
12     return userNums
13
14 def setRNumbers():
15     global randNums
16
17     randNums = random.sample(range(1, 46), 6)
```

```
18
19 def getRNumbers():
20     return randNums
21
22 def compareNumbers():
23     global userNums
24     global randNums
25     global collect
26
27     collect = []
28     for item in userNums:
29         if randNums.count(item) != 0:
30             collect.append(item)
31
32     return collect
```

II 빅데이터 처리를 위한 파이썬 프로그래밍

3 자주 활용하는 제공 모듈

실행 파일

ch12_appEx_01.py

```
01 import lottoMachine as lm
02
03 nums = []
04
05 print('1부터 45까지의 정수 6개를 입력하세요. ')
06 nums.append(int(input('Number 1 : ')))
07 nums.append(int(input('Number 2 : ')))
08 nums.append(int(input('Number 3 : ')))
09 nums.append(int(input('Number 4 : ')))
10 nums.append(int(input('Number 5 : ')))
11 nums.append(int(input('Number 6 : ')))
12
13 lm.setUNumbers(nums)      # 사용자 번호 설정
14 lm.setRNumbers()         # 랜덤 번호 설정
15
16 print('이번주 로또 번호', lm.getRNumbers())
17 print('내가 선택한 번호', lm.getUNumbers())
18 print('일치하는 숫자 :', lm.compareNumbers())
```

Q & A

